COMP 018: Database Administration — Activity 5 Progress Report

Database Maintenance and Monitoring

Monitoring Database Health, Maintaining Reliability, and Analyzing Performance

---

Cover Page & Metadata

Field	Details
Course	COMP 018 — Database Administration
Activity	Activity 5: Database Maintenance and Monitoring
Evaluating Group	**Group 7:** Jefferson Orpesa (Lead), Alba, Arreza, Robiso, Santos
Repository URL	https://github.com/jeffersonorpesa2-ops/grp07-campussafe
Maintenance Root Directory	https://github.com/jeffersonorpesa2-ops/grp07-campussafe/tree/main/db-maintenance
Database Platform	PostgreSQL 18.6 on x86_64 Windows (Compiled by MSVC)
Class Google Sheets Tracker	Tab: `Activity 5 Log`

---

1. Projects Monitored & Inspection Accounts

Group 7 monitored and maintained the PostgreSQL databases of all 8 peer group applications initialized in Activities 3 and 4:

Target Group	Project Name	Target Database	Accounts & Roles Used	Health Status	Evidence Folder
Group 2	Park-It: Parking Management	`grp_parkitdb`	`parkit_app` (Runtime), `parkit_dba` (DBA)	Healthy (9,128 kB, 99.9% Cache Hit)	`db-maintenance/group-02-parkit/`
Group 3	Project OJT RMA	`ojt_managementdb`	`ojt_app` (Runtime), `ojt_dba` (DBA)	Healthy (9,312 kB, 99.8% Cache Hit)	`db-maintenance/group-03-ojt-rma/`

Target Group	Project Name	Target Database	Accounts & Roles Used	Health Status	Evidence Folder
Group 4	BrewPOS: Point of Sale	`grp_brewposdb`	`brewpos_app` (Runtime), `brewpos_dba` (DBA)	Healthy (9,096 kB, 99.9% Cache Hit)	`db-maintenance/group-04-brewpos/`
Group 5	Lost & Found System	`grp05_lostandfounddb`	`lostandfound_app` (Runtime), `postgres`	Healthy (9,180 kB, 99.85% Cache Hit)	`db-maintenance/group-05-lostandfound/`
Group 6	HabalGo: Ride Hailing	`grp00_habalgodb`	`habalgo_app` (Runtime), `postgres`	Healthy (9,144 kB, 99.9% Cache Hit)	`db-maintenance/group-habalgo/`
Group 8	bwisIT Campus Canteen	`grp08_canteendb`	`canteen_app` (Runtime), `canteen_admin`	Healthy (9,250 kB, 99.9% Cache Hit)	`db-maintenance/group-08-canteen/`
Group 10	Binary Sisters Restaurant	`grp_binarysisters_projectdb`	`rors_app` (Runtime), `rors_maint`, `rors_dba`	Healthy (9,340 kB, 99.92% Cache Hit)	`db-maintenance/group-10-restaurant/`
Group 11	BoardTrack: Rental System	`grp11_boardtrackdb`	`boardtrack_app` (Runtime), `boardtrack_dba`	Healthy (9,182 kB, 99.97% Cache Hit)	`db-maintenance/group-11-boardtrack/`

---

2. Health & Activity Inspection Evidence

Across all 8 databases, standard diagnostic queries were executed to inspect storage footprint, buffer cache utilization, and connection concurrency:

2.1 Storage Footprint & Buffer Cache Hit Ratio

```
SELECT pg_size_pretty(pg_database_size(current_database())) AS database_size;

SELECT datname, blks_read, blks_hit,
       ROUND((blks_hit::numeric / NULLIF(blks_hit + blks_read, 0)::numeric) * 100, 2) AS
cache_hit_ratio_pct
FROM pg_stat_database WHERE datname = current_database();
```

- **Observed Metrics:** All databases maintained compact physical disk footprints (9,096 kB to 9,340 kB).

- **Cache Hit Ratio:** Ranged between **99.80% and 99.97%**, indicating that virtually all table pages and index blocks were served directly from PostgreSQL shared memory buffers (`shared_buffers`) without bottlenecking disk I/O.

2.2 Table Statistics & Bloat Inspection (`pg_stat_user_tables`)

```
SELECT relname, n_live_tup, n_dead_tup, last_vacuum, last_analyze, vacuum_count, analyze_count
FROM pg_stat_user_tables ORDER BY n_live_tup DESC;
```

- **Bloat Detection:** Identified significant dead tuple accumulation in mutable operational tables:
- `BoardTrack` (Group 11): 35 dead tuples in `room` caused by automated `sync_room_status()` triggers firing on lease insertions.
- `Restaurant` (Group 10): 24 dead tuples in `restaurant_table` from seat occupancy toggling.
- `Campus Canteen` (Group 8): 18 dead tuples in `orders` from status transitions.

---

3. Server Log Review & Issue Assessment

Inspection of the PostgreSQL server logs (`D:\Postgre\data\log\postgresql-*.log`) confirmed: 1. **Access Control Enforcement:** All expected security denials executed during Activity 4 testing were faithfully captured by the PostgreSQL logger with statement text (e.g. `ERROR: permission denied for table payment`, `FATAL: permission denied for database "grp07_campussafedb"`). 2. **Maintenance Denial Validation:** Attempting to execute `VACUUM` under runtime accounts (`boardtrack_app`, `canteen_app`, `rors_app`) logged warnings confirming proper denial: `WARNING: permission denied to vacuum "...", skipping it` 3. **Checkpoint Behavior:** Background checkpoints completed with normal write times (average sync time < 0.15s), verifying clean write-ahead logging (WAL) without I/O stalls.

---

4. Routine Maintenance Operations (`ANALYZE` & `VACUUM`)

To remediate dead tuple bloat and refresh query planner statistics:

1. **Planner Statistics Update (`ANALYZE VERBOSE`):**
2. Refreshed page estimates, tuple counts, and column distribution histograms (`pg_statistic`).
3. **Space Defragmentation (`VACUUM (VERBOSE, ANALYZE)`):**
4. Reclaimed dead tuple space into the Free Space Map (FSM) and updated Visibility Maps (VM).
5. Post-maintenance inspection confirmed **0 dead tuples** (`n_dead_tup = 0`) across all tables in all 8 project databases.

---

5. Query Performance Profiling (`EXPLAIN (ANALYZE, BUFFERS)`)

For each project, domain-critical reporting queries were evaluated before and after targeted index creation:

Target System	Profiled Workload / Query	Baseline Execution Time	Optimization Deployed	Optimized Execution Time	Performance Gain
BoardTrack (Group 11)	Active lease arrears & pending utility bills	1.049 ms (Buffers hit: 12)	Partial index on active leases (`idx_lease_active`) + composite payment status index	**0.833 ms** (Buffers hit: 12)	**+20.6% faster**
BoardTrack (Group 11)	Property occupancy & contracted revenue	0.429 ms (Buffers hit: 7)	Hash join on indexed `property_id`	**0.429 ms** (Optimal)	Sub-millisecond
Restaurant (Group 10)	Active orders & kitchen line items	0.784 ms	Composite index `(order_id, status)`	**0.462 ms**	**+41.1% faster**
Campus Canteen (Group 8)	Pre-order queue & customer order details	0.651 ms	Partial index `WHERE status = 'PENDING'`	**0.398 ms**	**+38.9% faster**
BrewPOS (Group 4)	Cashier shift totals & completed sales	0.812 ms	Index on `payment(order_id, status)`	**0.510 ms**	**+37.2% faster**
Park-It (Group 2)	Real-time zone occupancy calculation	0.540 ms	Covering index on `parking_record(zone_id)`	**0.320 ms**	**+40.7% faster**

---

6. Prioritized Recommendations & Technical Justification

1. **Deploy Partial Indexes on Active/Pending States:**
2. *Technical Justification:* In transactional systems, completed/expired records outnumber active ones by orders of magnitude. Partial indexes (`WHERE status = 'ACTIVE'`) reduce index size by over 80% and keep critical lookups in memory.
3. **Automate Autovacuum Scale Factors on Triggered Tables:**
4. *Technical Justification:* Tables like `room` or `restaurant_table` updated via automated triggers should have `autovacuum_vacuum_scale_factor = 0.05` to prevent MVCC bloat between scheduled maintenance windows.
5. **Dedicated Maintenance Role:**
6. *Technical Justification:* Maintain operational security by assigning maintenance duties to dedicated operator accounts holding `pg_maintain` and `pg_monitor` rather than using superusers.

---

7. Responsible AI Usage Declaration

- **AI Tools Utilized:** Google DeepMind Advanced Agentic AI Assistant (Antigravity).
- **Scope of AI Assistance:**
- Automated analysis of `pg_stat_user_tables` metrics and bloat identification.
- Extraction and formatting of `EXPLAIN (ANALYZE, BUFFERS)` query plan trees.
- Synthesizing index recommendations and maintenance SQL scripts.
- **Human Verification & Operational Control:** All `VACUUM`, `ANALYZE`, and index creation commands were physically executed on the local PostgreSQL 18.6 instance, and query performance metrics were captured live by Group 7 (Jefferson Orpesa, Alba, Arreza, Robiso, Santos).